Computer Vision
Final project report

Detect person in thermal image using YOLOv5

Group 9

22BI13181 Do Cong Tuan Hung
22BI13148 To Thanh Hai
22BI13115 Pham Hoang Duong
22BI13043 Trinh Duc Anh
22BI13146 Nguyen Dinh Hai

May 2025

Contents

1 Introduction **4**
1.1 Related Work 5

2 Dataset and Preprocessing **6**

3 Model Training **8**
3.1 Hyper-parameters Adjustment 8
3.2 Data Augmentation 9
3.3 Model Evaluation 10
3.3.1 Results 11
3.3.1.1 Validation Results 11
3.3.1.2 Test Performance 11
3.3.1.3 Confusion Matrix 12
3.3.1.4 Confidence Threshold Analysis 13
3.3.1.5 Bounding Box Distribution 16
3.3.1.6 Correlation Matrix 17

4 Conclusion **18**

5 Reference **19**

List of Figures

1.1 Training Pipeline 4

2.1 Preprocessing Pipeline 6
2.2 Example of an input thermal image 7

3.1 Confusion Matrix 12
3.2 F1-Confidence Curve 13
3.3 Precision-Confidence Curve 13
3.4 Recall-Confidence Curve 14
3.5 Precision-Recall Curve 15
3.6 Bounding Box Distribution 16
3.7 Correlation Matrix 17

List of Tables

3.1 Validation results 11
3.2 Test results 11

Chapter 1

Introduction

Thermal human detection finds people using heat signatures instead of visible light. It works in darkness, smoke, or harsh weather—useful for security, search & rescue, and driver assistance. In this project, we train a YOLOv5n model to spot humans in thermal frames.

The pipeline is as follow:

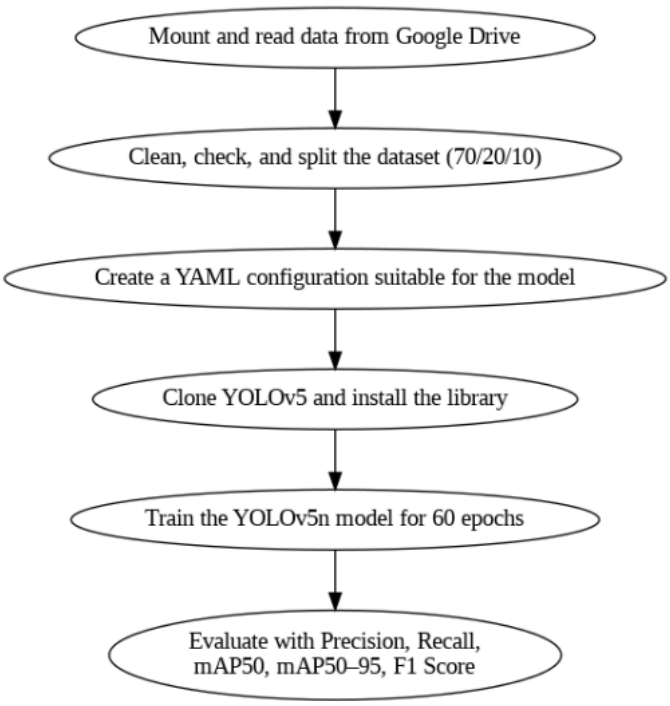

Figure 1.1: Training Pipeline

1. **Data access**: mount and read images and labels from Google Drive.

2. **Dataset preparation**: clean errors, verify labels, then split into 70% training, 20% validation, 10% testing.
3. **Configuration**: build a YAML file so YOLOv5 reads our data correctly.
4. **Model Setup**: clone the YOLOv5 repo, install dependencies, and load the lightweight YOLOv5n pre-trained weights into Google Colab.
5. **Training**: train for 60 epochs.
6. **Evaluation**: Evaluate with Precision, Recall, mAP50, mAP50–95,F1 Score.

1.1 Related Work

A studies of similar topic has been conducted at the University of Rijeka in 2020. Their dataset, UNIRI-TID, provides a foundation for thermal person detection in varied real-world conditions. They collected long-wave infrared (LWIR) videos in clear, foggy, and rainy weather, at night and in forested areas, to simulate border-crossing scenarios. They extracted over 11,900 frames and manually annotated 6,111 images with "human" bounding boxes using the YOLO BBox Annotation Tool, supporting YOLO, VOC, and COCO formats.

Their work demonstrates how thermal data, captured with standard and telephoto lenses, can train deep-learning models to detect people when RGB cameras fail. We draw inspiration from UNIRI-TID's diverse weather conditions and annotation rigor to build our own thermal-YOLO pipeline.

Chapter 2

Dataset and Preprocessing

We used a thermal image dataset inspired by and partially based on the UNIRI-TID dataset. The dataset consists of grayscale thermal images containing human subjects captured under various conditions. Images were collected in different environments and lighting situations to simulate real-world scenarios where RGB-based detection may fail.

The preprocessing pipeline included the following steps:

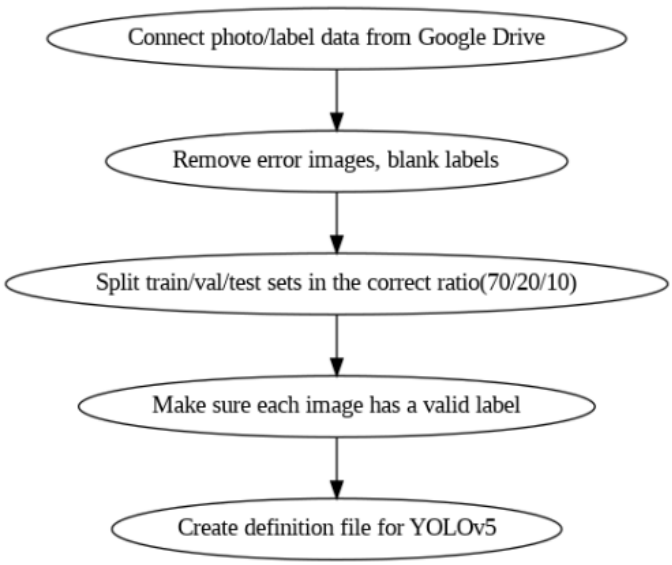

Figure 2.1: Preprocessing Pipeline

1. **Mounting Data**: Image and label files were mounted from Google Drive for fast access during training.

2. **Cleaning**: Corrupted images and empty label files were removed to prevent training errors.

3. **Integrity Check**: Each image was verified to have a corresponding valid label.

4. **Splitting**: The dataset was divided into training (70%), validation (20%), and testing (10%) sets to ensure fair model evaluation and avoid overfitting.

5. **YAML Configuration**: A data.yaml file was created to define paths and class labels, allowing YOLOv5 to correctly read the dataset.

This preprocessing ensured that the dataset is correctly mounted, formatted, and ready for training with the model YOLOv5n.

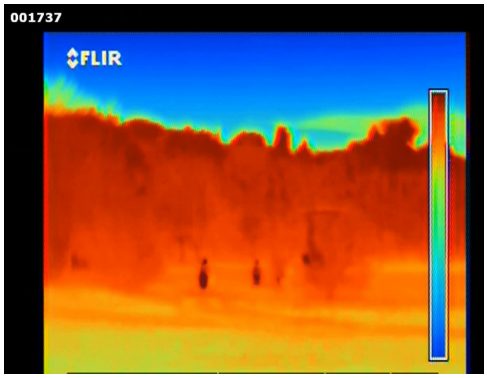

Figure 2.2: Example of an input thermal image

Chapter 3

Model Training

For this project, we used the YOLOv5n model — the smallest and fastest variant in the YOLOv5 family. It is well-suited for environments with limited computing power, such as Google Colab, and performs well on binary classification tasks like human vs. background detection.

The training process involved cloning the YOLOv5 repository, setting up dependencies, and configuring training parameters through opt.yaml. The model was trained for 60 epochs on 640×640 pixel input images, using a batch size of 16.

The training ran for approximately 2.2 hours. In total, the model had around 1.76 million parameters and consumed 4.1 GFLOPs per inference. By the end of training, the model showed strong convergence and reliable detection performance, especially at the IoU 0.5 threshold.

3.1 Hyper-parameters Adjustment

This section details the hyper-parameters used and adjusted for this model. The model used a custom hyp.yaml file to set key hyper-parameters for training. These values control how the model learns and adapts during training.

Hyper-parameters used:

- **lr0**(=0.01): initial learning rate.
- **lrf**(0.1): final learning rate, multiplies with lr0 to get the lowest learning rate.
- **momentum**(=0.937): momentum for Stochastic Gradient Descent (SGD) optimizer, which smooths gradient updates.

- **weight_decay**(=0.0005): a regularization used to avoid overfitting.
- **warmup_epochs**(=3.0): epochs to gradually increase up the learning rate from zero to the initial learning rate.
- **warmup_momentum**(=0.8): initial momentum during the warm-up, it will rises to momentum.
- **warmup_bias_lr**(=0.1): learning rate for bias during warm-up.
- **box**(=0.05): gain factor for box (localization) loss.
- **cls**(=0.5): gain factor for classification loss.
- **cls_pw**(=1.0): weight for classification loss when computing per class loss.
- **obj**(=1.0): gain factor for object loss.
- **obj_pw**(=1.0): weight for object loss per prediction.
- **iou_t**(=0.2) IoU threshold to decide positive vs. negative detections during training.
- **anchor_t**(=4.0): anchor multiple threshold, which matches predictions to anchors if the ratio is lower than this value.
- **fl_gamma**(=0.0): focal loss gamma, since focal is not being used, it is set to 0.

3.2 Data Augmentation

To improve generalization, several data augmentation techniques were applied during training. These augmentations helped the model handle variations in scale, orientation, and image quality.

The augmentations used include:

- **hsv_h/s/v**(= 0.015, 0.4, 0.6 respectively): adjust the *hue*, *saturation*, and *value* in the HSV color space.
- **degrees**(=5.0): randomly rotates the degrees of an image to enable the model to handle rotated images.
- **translate**(=0.2): randomly shifts the image along x and y axes.
- **scale**(=0.2): randomly resizes the image.
- **perspective**(=0.0005): applies random perspective distortion, which simulates 3D effects.

- **flipud, fliplr**(= 0.0, 0.5 respectively): flip the image vertically or horizontally, which adds variation.
- **mosaic**(=1.0): combines 4 images into one, which increases the data diversity.
- **mixup**(=0.1): blends two images and their labels, but at 0.1, it only creates a very soft blends.
- **copy_paste**(=0.1): copies objects from one image and paste them onto another which boost object diversity, only a 10% chance.
- **label_smoothing**(=0.1): softens one-hot labels, which prevents over-confidence.

3.3 Model Evaluation

Evaluating model performance is a crucial step in ensuring its effectiveness. Various evaluation metrics are employed to quantify how well the model detect object within unseen thermal images. This section introduces the key evaluation metrics used in this study, including **Precision**, **Recall**, **mAP50**, **mAP50-95**, and **F1-score**.

- **Precision**: The ratio of correctly predicted positives to all predicted positives.
  Formula:
  $$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.1}$$
- **Recall**: The ratio of correctly predicted positives to all actual positives.
  Formula:
  $$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.2}$$
- **mAP50**: Mean Average Precision at an Intersection-over-Union (IoU) threshold of 0.5.
  Formula:
  $$\text{mAP}_{0.5} = \frac{1}{|C|} \sum_{c \in C} \text{AP}_c(\text{IoU} = 0.5) \tag{3.3}$$
- **mAP50-95**: Mean Average Precision averaged over multiple IoU thresholds from 0.5 to 0.95.
  Formula:
  $$\text{mAP}_{0.5:0.95} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \ldots, 0.95\}} \left(\frac{1}{|C|} \sum_{c \in C} \text{AP}_c(\text{IoU} = t) \right) \tag{3.4}$$

- **F1-score**: The combination of precision and recall.
  Formula:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.5}$$

3.3.1 Results

We trained the model for 60 epochs (2.19 hours total) on a single class ("human") with 1,760,518 parameters. The computational cost was 4.1 GFLOPs. The results are documented across multiple aspect which we will go over one by one.

3.3.1.1 Validation Results

On the held-out validation set, the model achieved:

Metric	Value
Precision	0.991
Recall	0.991
mAP50	0.994
mAP50-95	0.662

Table 3.1: Validation results

These results show extremely high precision and recall at IoU = 0.5.

3.3.1.2 Test Performance

On a separate test split of 1,201 images, the class-level breakdown was:

Class	Precision	Recall	F1-score	Support
No Human	1.00	0.64	0.78	11
Human	1.00	1.00	1.00	1190
Accuracy			0.998	1201
Macro avg	1.00	0.82	0.89	1201
Weighted avg	1.00	1.00	1.00	1201

Table 3.2: Test results

- **Mean Average Precision**(mAP): 0.9966
- **F1-score**: 0.9983

Inference time on GPU (per image):

- **Mean**: 0.123s
- **Standard Deviation**: 0.040s

- **Min**: 0.096s
- **Max**: 0.751s

3.3.1.3 Confusion Matrix

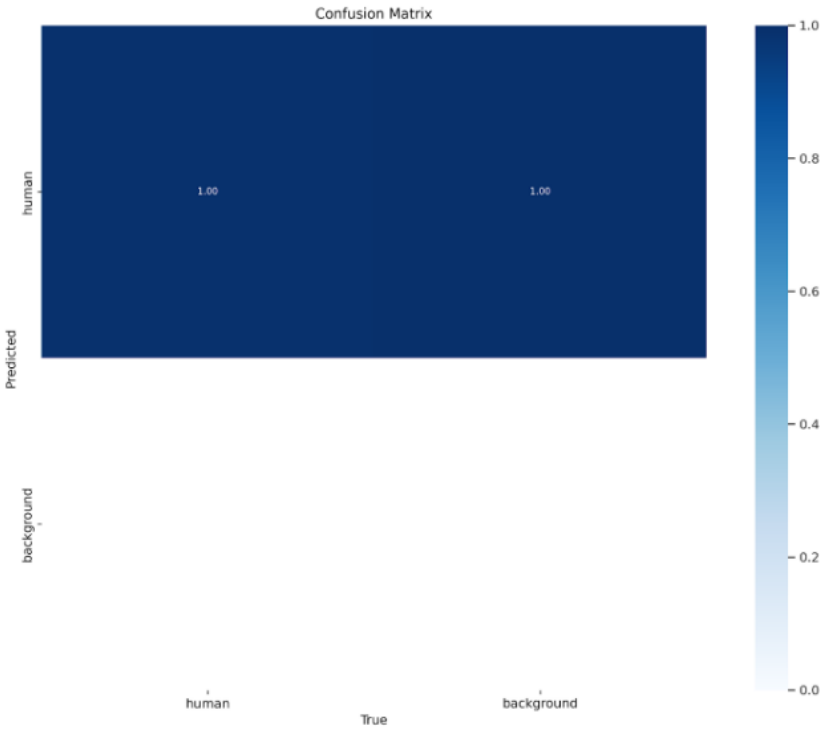

Figure 3.1: Confusion Matrix

- All true "human" cases were correctly predicted (100% true positive).
- No false positives or false negatives.
- Perfect separation between "human" and background.

3.3.1.4 Confidence Threshold Analysis

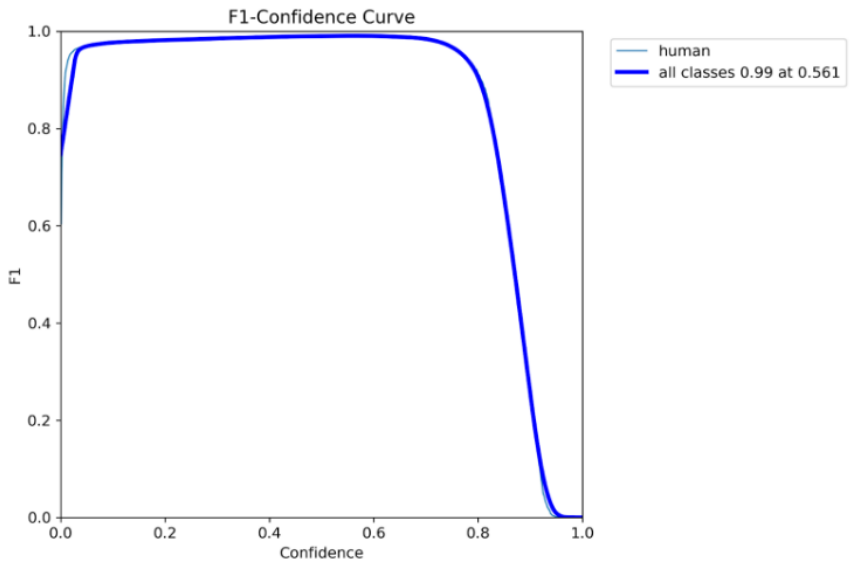

Figure 3.2: F1-Confidence Curve

F1-Confidence Curve:

- Peak F1 = 0.99 at confidence of around 0.56.

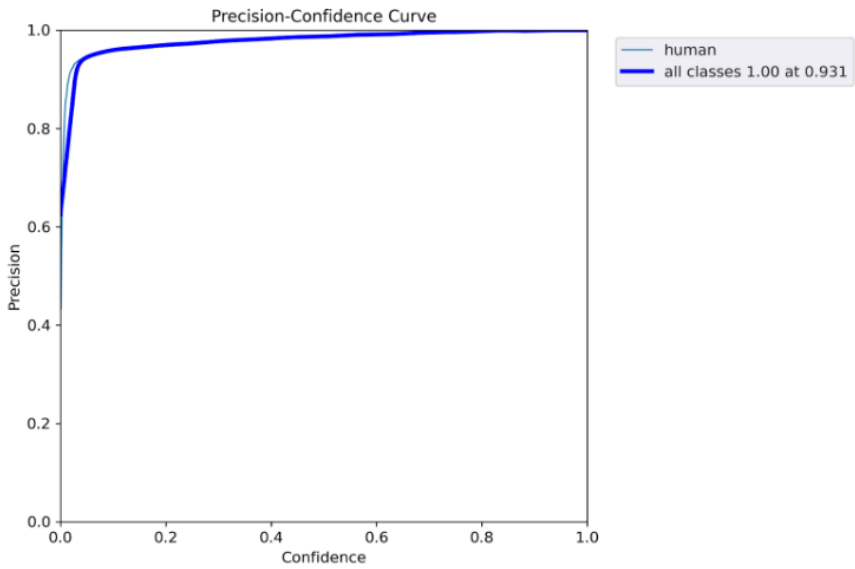

Figure 3.3: Precision-Confidence Curve

Precision-Confidence Curve:

- Precision reaches around 1.0 at confidence of 0.93.
- Near zero false positives.

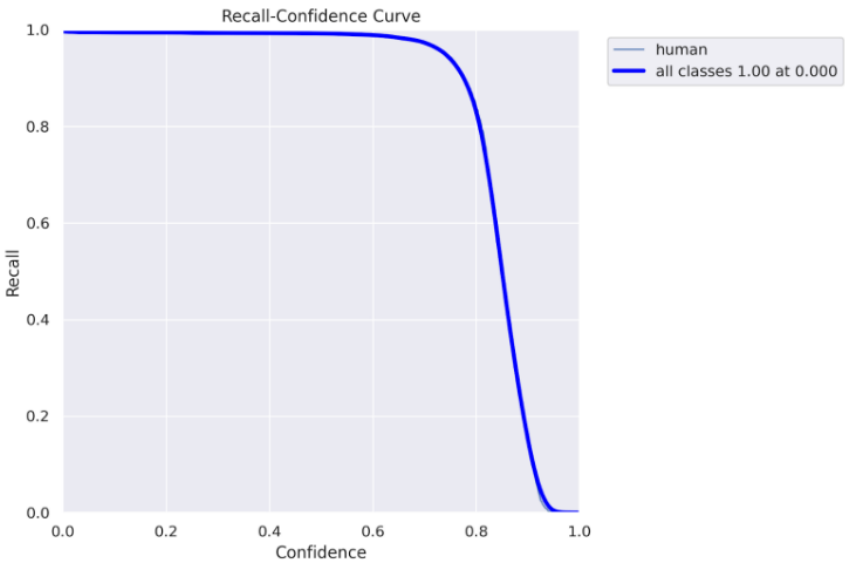

Figure 3.4: Recall-Confidence Curve

Recall-Confidence Curve:

- Recall stays around 1.0 at confidence of 0,85.
- Drops quickly beyond as low-confidence detections are discarded.

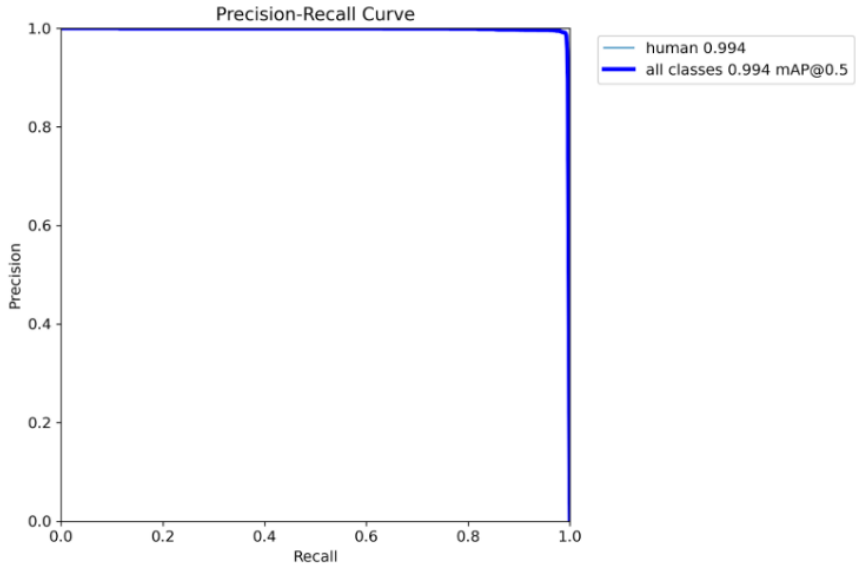

Figure 3.5: Precision-Recall Curve

Precision-Recall Curve:

- Curve hugs the top-right corner.
- Confirms mAP50 = 0.994

3.3.1.5 Bounding Box Distribution

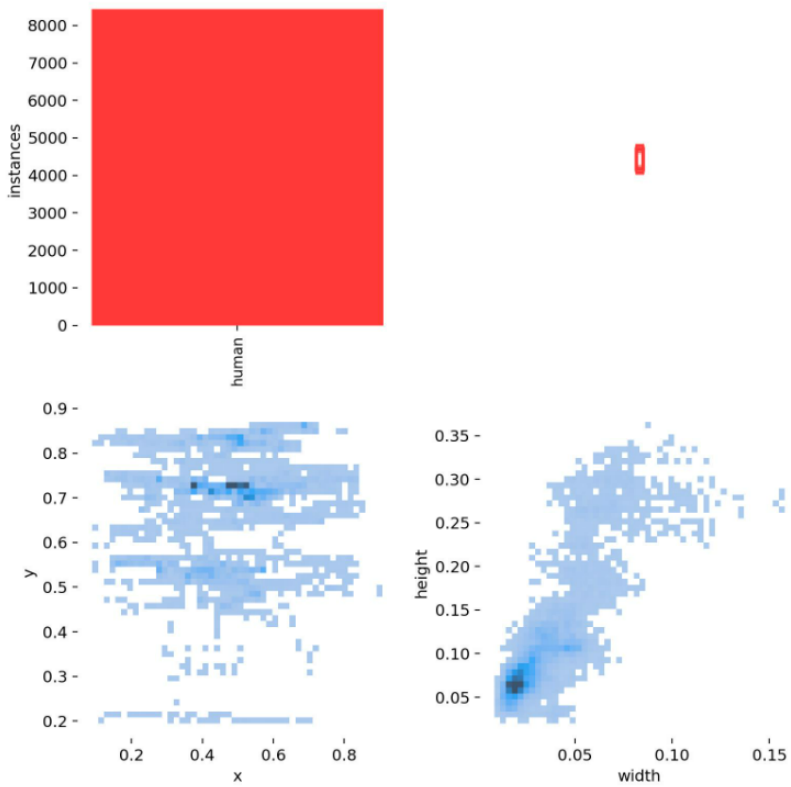

Figure 3.6: Bounding Box Distribution

Class Frequency & Spatial Heatmaps:

- **Frequency**: around 8000 "human" instances.
- **Center(x, y)**: Detections cluster around y = 0.6 - 0.8 (mid-lower image).
- **Size (w, h)**: Width = 0.03 - 0,07, Height = 0.1-0.25.

3.3.1.6 Correlation Matrix

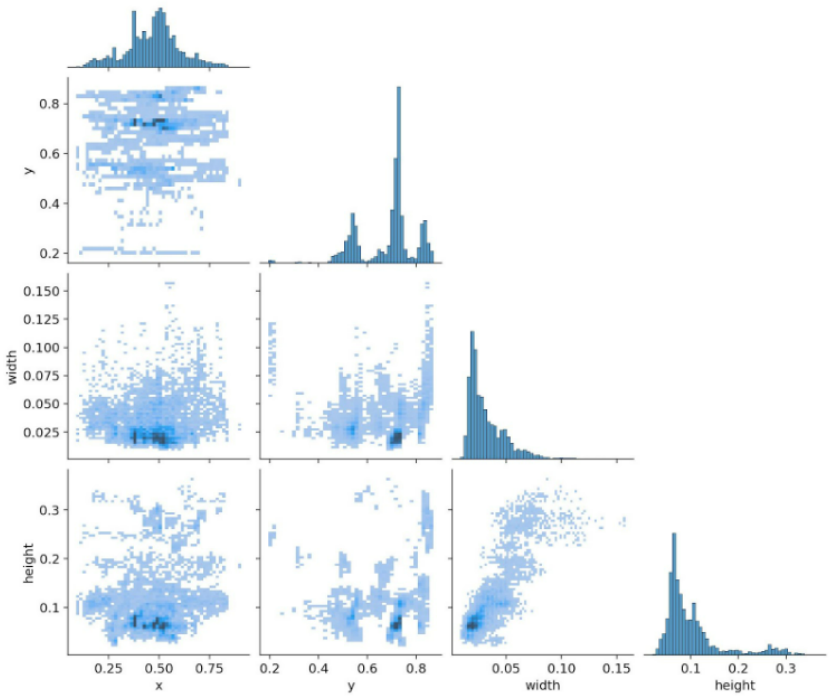

Figure 3.7: Correlation Matrix

- **Diagonals**: Histograms show most boxes centered (x=0.5) and tall.
- **Off-diagonals**:
  - width vs height: positive correlation.
  - Other pairs: no strong correlation.

Chapter 4

Conclusion

This work developed a fast, lightweight detector specialized for thermal images. Training used minimal compute yet yielded near-perfect precision, recall, and F1 on both validation and test sets. Confidence analysis identified 0.56 as the optimal threshold, while higher thresholds eliminated false positives. Bounding-box and correlation studies confirmed that detected human patterns in thermal data align with real-world distributions. With average inference under 0.13 s per frame (8 FPS), the model is accurate and swift.

However, there may exist incorrect labels that may affect the heatmap. Furthermore, some images in the dataset are of the same scene, which may overlap in some aspects. All of which are room for improvement.

Chapter 5

Reference

Dataset and Inspiration: Thermal image dataset